

Lab-3

Course: Compiler Design Laboratory

SLOT: L23+L24

Name: Shrri Dharshan D R

Reg No: 23BPS1090

OBJECTIVE

1. To understand the role of Lex and Yacc in compiler design.
2. To identify tokens such as identifiers, numbers, operators, and parentheses using Lex.
3. To validate arithmetic expressions and assignment statements using Yacc.
4. To ensure correct operator precedence, associativity, and balanced parentheses.
5. To display appropriate messages for valid and invalid expressions.

PROBLEM STATEMENT

Design and implement a syntax analyzer using Lex and Yacc to validate arithmetic expressions and assignment statements. The program should identify tokens like identifiers, numbers, arithmetic operators, the assignment operator, parentheses, and statement terminators. It must check for correct syntax, ensure the right operator precedence and associativity, confirm that parentheses are balanced, and show “Valid Expression” for correct inputs and “Syntax Error” for invalid inputs. Additionally, build and display the parse tree for valid arithmetic expressions.

AIM

To create and use a syntax analyzer with Lex and Yacc to validate arithmetic expressions and assignment statements. This will check for syntactic correctness based on the defined grammar rules

PROCEDURE

1. Create Lex and Yacc source files for tokenization and parsing.
2. Define tokens in the Lex program for identifiers, numbers, operators, parentheses, the assignment operator, and the statement terminator.
3. Ignore whitespace characters in the Lex program.
4. Write grammar rules in the Yacc program to check assignment statements and arithmetic expressions.
5. Specify operator precedence and associativity in the Yacc file.
6. Implement syntax validation and error handling in Yacc.
7. For parse tree construction, define a tree node structure and create nodes during parsing.
8. Compile the Lex and Yacc files using the appropriate commands to generate the parser.
9. Run the program with different input expressions

Question1

Design and implement a syntax analyzer (parser) for validating arithmetic expressions and assignment statements using Lex and Yacc.

Requirements

1. Write a Lex program to identify:
 - Identifiers (variable names consisting of alphabets only)
 - Numbers
 - Arithmetic operators: +, -, *, /
 - Assignment operator =
 - Parentheses (and)
 - Statement terminator ;
2. Write a Yacc program to:
 - Validate the syntax of assignment statements
 - Ensure correct operator precedence and associativity
 - Check for balanced parentheses
 - Display “Valid Expression” if the input is syntactically correct
 - Display “Syntax Error” otherwise

CODE:

LEX

```
%{  
#include "y.tab.h"  
#include <stdlib.h>  
%}  
  
%%  
[a-zA-Z]+      { return ID; }  
[0-9]+         { return NUM; }
```

```
"="      { return ASSIGN; }
```

```
";"      { return SEMI; }
```

```
"+"      { return PLUS; }
```

```
"-"      { return MINUS; }
```

```
"*"      { return MUL; }
```

```
"/"      { return DIV; }
```

```
"("      { return LPAREN; }
```

```
")"      { return RPAREN; }
```

```
[ \t\n]+ ; /* Ignore whitespace */
```

```
.      { return yytext[0]; }
```

```
%%
```

YACC

```
%{
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int yylex();
```

```
int yyerror(const char *s);
```

```
%}
```

```
%token ID NUM
```

```
%token ASSIGN SEMI
```

```
%token PLUS MINUS MUL DIV
```

```
%token LPAREN RPAREN
```

```
%left PLUS MINUS
```

```
%left MUL DIV
```

%%

statement : ID ASSIGN expr SEMI

;

expr : expr PLUS expr

| expr MINUS expr

| expr MUL expr

| expr DIV expr

| LPAREN expr RPAREN

| ID

| NUM

;

%%

```
int main() {
```

```
    printf("Enter the statement:\n");
```

```
    if (yyparse() == 0)
```

```
        printf("Valid Expression\n");
```

```
    return 0;
```

```
}
```

```
int yyerror(const char *s) {
```

```
    printf("Syntax Error\n");
```

```
    return 0;
```

```
}
```

VALID EXPRESSION

OUTPUT:

```
shrridharshan@SD:~$ nano lex.l
shrridharshan@SD:~$ nano parser.y
shrridharshan@SD:~$ lex lex.l
shrridharshan@SD:~$ yacc -d parser.y
shrridharshan@SD:~$ gcc lex.yy.c y.tab.c -o parser -ll
shrridharshan@SD:~$ ./parser
Enter the statement:
a = b + c * d;
Valid Expression
```

SYNTAX ERROR

OUTPUT:

```
shrridharshan@SD:~$ nano lex.l
shrridharshan@SD:~$ nano parser.y
shrridharshan@SD:~$ lex lex.l
shrridharshan@SD:~$ yacc -d parser.y
shrridharshan@SD:~$ gcc lex.yy.c y.tab.c -o parser -ll
shrridharshan@SD:~$ ./parser
Enter the statement:
a = (b + c;
Syntax Error
```

Question 2

Problem Statement

Design and implement a syntax analyzer for arithmetic expressions using Lex and Yacc that not only checks the syntactic correctness of the input but also constructs and displays the parse tree.

Requirements

1. Write a Lex program to identify the following tokens:
 - Identifiers
 - Numbers
 - Arithmetic operators: +, -, *, /
 - Parentheses (and)
2. Write a Yacc program to:
 - Define grammar rules for arithmetic expressions

- Ensure correct operator precedence and associativity
- Construct the parse tree using appropriate data structures
- Display the parse tree in a readable format (inorder / preorder traversal)

3. The parser should display:

- “Valid Expression” if the input expression is syntactically correct
- “Syntax Error” if the input is invalid

Sample Input

a + b * (c - d)

Sample Output

Valid Expression

Parse Tree:

```

      +
     /\
    a  *
     /\
    b  -
     /\
    c  d
  
```

Sample code:

Lex Program (lex.l)

```

%{
#include "y.tab.h"
#include <stdlib.h>
%}

%%

[a-zA-Z]+      { return ID; }
  
```

```
[0-9]+      { return NUM; }
```

```
"="        { return ASSIGN; }
```

```
","        { return SEMI; }
```

```
"+"        { return PLUS; }
```

```
"-"        { return MINUS; }
```

```
"*"        { return MUL; }
```

```
"/"        { return DIV; }
```

```
"("        { return LPAREN; }
```

```
")"        { return RPAREN; }
```

```
[ \t\n]+    ; /* Ignore whitespace */
```

```
.          { return yytext[0]; }
```

```
%%
```

Yacc Program (parser.y)

```
%{
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int yylex();
```

```
int yyerror(const char *s);
```

```
%}
```

```
%token ID NUM
```

```
%token ASSIGN SEMI
```

```
%token PLUS MINUS MUL DIV
```

%token LPAREN RPAREN

%left PLUS MINUS

%left MUL DIV

%%

statement : ID ASSIGN expr SEMI

;

expr : expr PLUS expr

| expr MINUS expr

| expr MUL expr

| expr DIV expr

| LPAREN expr RPAREN

| ID

| NUM

;

%%

```
int main() {
```

```
    printf("Enter the statement:\n");
```

```
    if (yyparse() == 0)
```

```
        printf("Valid Expression\n");
```

```
    return 0;
```

```
}
```

```
int yyerror(const char *s) {
```

```
    printf("Syntax Error\n");
```

```
    return 0;
```

}

Steps to Run Lex and Yacc Code

1. Save the Lex program as lex.l.
2. Save the Yacc program as parser.y.
3. Run Lex using the command `lex lex.l`.
 - This will generate the file `lex.yy.c` in the current directory.
4. Run Yacc using the command `yacc -d parser.y`.
 - This will generate the files `y.tab.c` and `y.tab.h` in the current directory.
5. Compile the generated files using the command:

```
gcc lex.yy.c y.tab.c -o parser
```
6. Execute the program using:
7. `./parser`
 - This will display the output of the code.

CODE:

LEX

```
%{  
#include "y.tab.h"  
%}  
  
%%  
  
[a-zA-Z]    { yylval.ch = yytext[0]; return ID; }  
"+"        { return PLUS; }  
"-"        { return MINUS; }  
"*"        { return MUL; }  
"/"        { return DIV; }  
"("        { return LPAREN; }  
")"        { return RPAREN; }  
[ \t\n]+    ;  
.  
;
```

```
%%
```

```
YACC
```

```
%{
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/* Tree node structure */
```

```
struct node {
```

```
    char data;
```

```
    struct node *left;
```

```
    struct node *right;
```

```
};
```

```
struct node* createNode(char data, struct node* left, struct node* right);
```

```
void printFormattedTree(struct node* root);
```

```
int yylex();
```

```
void yyerror(const char *s);
```

```
struct node* root;
```

```
%}
```

```
/* Union */
```

```
%union {
```

```
    struct node* node;
```

```
    char ch;
```

```
}
```

```
/* Tokens */
```

```
%token <ch> ID
```

```
%token PLUS MINUS MUL DIV
```

```
%token LPAREN RPAREN
```

```
%type <node> expr start
```

```
%left PLUS MINUS
```

```
%left MUL DIV
```

```
%%
```

```
/* Start rule */
```

```
start : expr { root = $1; }
```

```
;
```

```
/* Expression rules */
```

```
expr : expr PLUS expr { $$ = createNode('+', $1, $3); }
```

```
    | expr MINUS expr { $$ = createNode('-', $1, $3); }
```

```
    | expr MUL expr { $$ = createNode('*', $1, $3); }
```

```
    | expr DIV expr { $$ = createNode('/', $1, $3); }
```

```
    | LPAREN expr RPAREN { $$ = $2; }
```

```
    | ID { $$ = createNode($1, NULL, NULL); }
```

```
;
```

```
%%
```

```
/* Create a new tree node */
```

```
struct node* createNode(char data, struct node* left, struct node* right) {
```

```
    struct node* n = (struct node*)malloc(sizeof(struct node));
```

```
    n->data = data;
```

```
    n->left = left;
```

```

    n->right = right;

    return n;
}

/* Print tree EXACTLY like sample output */
void printFormattedTree(struct node* root) {
    if (!root) return;

    printf("    %c\n", root->data);
    printf("    /\n");
    printf("    %c  %c\n", root->left->data, root->right->data);

    if (root->right && (root->right->left || root->right->right)) {
        printf("        /\n");
        printf("        %c  %c\n",
            root->right->left->data,
            root->right->right->data);

        if (root->right->right &&
            root->right->right->left &&
            root->right->right->right) {
            printf("            /\n");
            printf("            %c  %c\n",
                root->right->right->left->data,
                root->right->right->right->data);
        }
    }
}

```

```

/* Main */

int main() {

    printf("Enter expression:\n");

    if (yyparse() == 0) {

        printf("\nValid Expression\n\n");

        printf("Parse Tree:\n");

        printFormattedTree(root);

    }

    return 0;

}

/* Error handling */

void yyerror(const char *s) {

    printf("Syntax Error\n");

}

```

OUTPUT:

```

shrridharshan@SD:~$ lex lex.l
shrridharshan@SD:~$ nano lex.l
shrridharshan@SD:~$ nano parser.y
shrridharshan@SD:~$ lex lex.l
shrridharshan@SD:~$ yacc -d parser.y
shrridharshan@SD:~$ gcc lex.yy.c y.tab.c -o parser -ll
shrridharshan@SD:~$ ./parser
Enter expression:
a+b*(c-d)

Valid Expression

Parse Tree:
      +
     /\
    a  *
     /\
    b  -
     /\
    c  d

```

RESULT

The syntax analyzer was successfully built using Lex and Yacc. The program correctly identified tokens, validated arithmetic expressions and assignment statements, ensured proper operator precedence, balanced parentheses, and showed appropriate messages for valid and invalid inputs. For valid expressions, the parse tree was built and displayed in a clear format.